20/10/26: I've finally starting learning how training works in YOLO and have set up a nano model of YOLO. Nano is not the model we are using, I just wanted something that would run fast, which is why I did it in a single epoch with a batch size of 3, since I would rather not leave it on for 10 hours to have it fail and not work. This test was mainly done to see if it was possible for me to do it on my personal computer and what it all means.
I also found out that by default YOLO trains on cpu, which is okay if I want to wait until next week for a model, but since I do not, I looked into how to use GPU for training. Considering that I am actively using my computer for other work as I train (having around 70 loaded tabs), training is not going terribly slow.
I am now going to read into what all the different parameters mean when it comes to training because I can infer what it means, but would like to be sure.

New terms (to me):

- MPS: Metal Performance Shaders, on a super high level offer low level access to GPU (in apple devices) for applications. It is a part of the Metal family (an API made by apple). Due to how the M series chips work, MPS is (from a very preliminary research) the best way to use GPU on apple silicon computers. Source: `https://developer.apple.com/documentation/metalperformanceshaders` and `https://en.wikipedia.org/wiki/Metal_(API)`
- Zero-shot results: Zero-shot learning is a training/fine-tuning method where The model is trained to infer on things it does not actually know what they look like from before, but rather it has some other semantic information about it, such as a description of what the object looks like.
  A zero-shot result I infer to mean that the model is infering on new images that it has never seen and validating how well it did based on a ground truth `https://www.ibm.com/think/topics/zero-shot-learning`
- Epoch:
- Batch: Divide our training data into smaller sub-sections so that training might go faster `https://www.geeksforgeeks.org/deep-learning/batch-size-in-neural-network/`
  I want to read this resource more in depth, there are some things towards the bottom that I want to understand better.

24/10/26: I've spent the past 4 days figuring out how to train the models, and actually training them. Turns out that it is slightly more complicated than I expected, not because of the code being hard (that part is somewhat easy), but rather because using the lab computers for training is not the most intutive, where I had to understand how docker (recommended by Mike Tie) works on a deeper level, since by default I could not actually install anything to the lab computer.
Thankfully after a couple hours of trying to get the container that Mike Tie recommended to work, I did some extra research and found that ultralytics provides a container that I can use without having to figure out how to install all dependencies. Figuring out how to properly use it was also a bit of a challenge, but not more than making a new container

from scratch (an endeavour that I was engaging in when I got the thought of "I wonder if ultralytics have released a container already").